

Proyecto 12 — LLM Data Engineering Pipeline

Versión ejecutiva del entregable

Curso: Diseño de Infraestructura Escalable — BSG Institute **Profesor:** Msc. PgP. Andrés Felipe Rojas Parra — Chief Artificial Intelligence Officer **Estudiante:** Alfredo Suárez — arse.alf@gmail.com **Fecha:** 2026-06-01

Caso de uso: Plataforma de Conocimiento del Marketplace B2B PyME de **Acme Co**, primer motor de instrumentación de la tesis Economic Graph de la PyME Mexicana de **Grupo Acme**.

Material de revisión

Tipo	Link	Para qué sirve
 Video walkthrough	https://www.loom.com/share/af5e82791ab546b9a6a2a3914e065047	Empezar aquí. Loom de ~10 min explicando contexto y cómo se construyó el proyecto
 Presentación detallada	https://gamma.app/docs/Proyecto-12-LLM-Data-Engineering-Pipeline-6wow0z7dqdyr7f9	Slides con caso de uso, stack, regla maestra CNBV, resultados
 Resumen ejecutivo	https://gamma.app/docs/Proyecto-12-LLM-Data-Engineering-Pipeline-auszjr6wb2as59u	Slides condensados para sponsor / comité
 Repositorio GitHub	https://github.com/AlfredoSuarez/bsg-final-project-data-llm-test_entregable	Código, IaC, docs, evidencia del deploy real (anonimizado)
 PDF completo (146 pp)	Docs Finales/Proyecto12_Entregable_Final.pdf en el repo	Documentación técnica completa para evaluación detallada
 Spec original BSG	Docs Finales/01_Proyecto12_Origen_BSG.md en el repo	El "qué se pidió" — rúbrica de 12 componentes

Resumen ejecutivo

Pipeline cloud-nativo en **AWS** que ingesta 500+ documentos del Marketplace B2B PyME de Acme Co (PDF/DOCX/HTML), los limpia, aplica **chunking semántico con Quality Gate** (incluyendo regla maestra CNBV para chunks financieros), genera embeddings con **Bedrock Titan V2 (1024 dim)**, los indexa en **Aurora Post-**

greSQL + pgvector con índice HNSW, y orquesta el flujo end-to-end con **Step Functions** + observabilidad completa en **CloudWatch**.

Cumple **12/12 componentes** de la rúbrica del Proyecto 12 (~92/100 pts directos) y entrega la **plomería de datos del Año 1** de un activo estratégico de 7 años para Acme Co / Grupo Acme.

¿Por qué AWS + Bedrock Titan V2?

AWS — 3 principios rectores

#	Principio	Implicación
1	Cero datos fuera del perímetro AWS	Bedrock vía VPC endpoint privado · Aurora en subnets privadas · sin tránsito por internet pública
2	Compliance LFPDPPP/CNBV/CONDUSEF/INAI estructural	Residencia de datos en región AWS · CloudTrail nativo · datos NO usados para entrenar modelos
3	Ecosistema Grupo Acme ya en AWS	Banco Acme, AcmeCo, Retail Acme ya operan en AWS · integración futura nativa

Bedrock Titan V2 vs alternativas

Dimensión	Titan V2 (elegido)	OpenAI 3-large	BGE-M3 self-hosted
Dimensiones	1024	3072	1024
Costo / 1M tokens	USD 0.02	USD 0.13 (6.5× más)	USD 0.03-0.15 (GPU idle)
Residencia datos	Dentro AWS	Sale a OpenAI	Control propio
Compliance LFPDPPP	Nativo	Tier Enterprise	Ops propias
Integración AWS	IAM + VPC + KMS	Externa, requiere NAT	ECS/EKS + GPU
Time-to-production	Horas	Días	Semanas

Decisión: Titan V2 gana en costo, compliance, integración nativa. Trade-off: calidad MTEB marginal vs OpenAI desaparece en evaluación humana real.

Justificación completa en `docs/00_decisiones_clave.md` del repo + sección equivalente del PDF completo (páginas 14-22).

Lo que se entregó

Componentes técnicos

- **VPC privada AWS** con 7 endpoints (S3, DDB, Bedrock, Secrets, ECR API/DKR, Logs)
- **Aurora PostgreSQL 16 Serverless v2** con pgvector + 11 índices (HNSW m=16 ef_construction=64)
- **Glue 4.0 Spark** ETL (PDF/DOCX/HTML → Parquet)
- **Lambda container arm64** chunking con Quality Gate + Bedrock Titan V2 embedding
- **ECS Fargate container arm64** indexer con pycpg2 batch UPSERT
- **Step Functions Standard** state machine con Map paralelo, Catch, Retry
- **DynamoDB** `index_versions` + `chunk_quality_audit` con SHA-256 dataset hash
- **CloudWatch** dashboard 13 widgets + 7 alarmas + SNS
- **Terraform 1.6+** IaC (16 archivos .tf, 83 recursos)

Validación

-  93/93 tests pytest verde
-  Docker builds arm64 (chunking + indexer) con healthcheck OK
-  Terraform plan vs AWS real: 83 recursos sin errores
-  **Deploy end-to-end SUCCEEDED en AWS** (`run-demo-20260601-015935` , 2 min 30s)
-  5 chunks indexados en Aurora con embeddings Titan V2 reales (1024 dim)
-  HNSW cosine search funcional: similarity 1.0 → 0.71 → 0.58 → 0.54 → 0.22
-  Regla maestra CNBV verificada: 100% chunks pass con marcador financiero detectado
-  Audit trail completo: 8/8 chunks decisionados en DDB con timestamp + reason + metrics_json

KPIs en 4 capas

Capa	Pregunta que responde	Indicadores	Estado
1. Técnica	¿El pipeline corre, qué tan rápido, qué tan limpio?	6 (throughput, latencia p95, Quality Gate %, cost embeddings, Recall@5)	 Instrumentados
2. Negocio	¿Sirve para el Marketplace? ¿Justifica costo?	4 (cobertura vertical, frescura, costo/consulta, tiempo ahorrado)	 Parcial
3. Compliance	¿Cumple LFPDPPP/CNBV/INAI?	4 (% financiero auditado, integridad version_id, rollback, CloudTrail)	 Instrumentados
4. Agente LLM	¿El agente que CONSUME el índice responde bien?	12 (Task Success, Faithfulness, Hallucination Rate, etc.)	 Fase 2 (roadmap)

Detalle en [docs/13_indicadores_y_justificacion.md](#) del repo + sección equivalente del PDF completo.

Hallazgos reales del deploy a AWS

Durante el deploy se encontraron y resolvieron **12 bugs reales** documentados en [docs/12_lecciones_aprendidas.md §2bis](#) :

Categoría	Bugs	Ejemplo
laC contra AWS API	5	Aurora 16 rechaza <code>vector</code> en <code>shared_preload_libraries</code> ; DDB tags no permiten paréntesis
Container + Lambda + ECS	4	Buildx attestations rompen Lambda; ECS Fargate en private subnet sin end-points ECR falla pull
Código Lambda	3	tiktoken descarga tokenizer desde Azure Blob (no funciona en VPC privada); doc_type del Parquet partition no se lee con pyarrow

Cada bug con causa raíz + fix aplicado. Es material valioso para futuras iteraciones del equipo.

Costo de operación

Concepto	Idle (24/7)	Por run end-to-end
Aurora Serverless v2 (0.5 ACU)	\$43/mes	\$0.01 surge
VPC endpoints (5 interface)	\$40/mes	\$0.001 data
CloudWatch + Secrets + ECR storage	\$9/mes	—
Glue Job (95 s, 4 workers G.1X)	\$0	\$0.047
Lambda chunking (4 invokes × 5s)	\$0	\$0.0007
ECS Fargate indexer (~50s)	\$0	\$0.001
Bedrock Titan V2 (~830 tokens)	\$0	\$0.0000166
Step Functions (~15 transitions)	\$0	\$0.0004
Total	~\$92/mes	~\$0.05 por run

Disclosure de anonimización

Por confidencialidad del cliente real, este entregable usa nombres anonimizados con un mapping consistente:

En este entregable	Tipo
Acme Co	Cliente principal (telco)
Grupo Acme	Conglomerado al que pertenece
Banco Acme	Subsidiaria bancaria
Retail Acme	Subsidiaria de retail
PyME Digital	ICP (perfil de la PyME representativa)
Aliado Telco	Programa de aliados comerciales

Carrier Billing se mantiene porque es nomenclatura estándar de la industria telco-fintech, no marca registrada.

Siguiente fase (roadmap)

Fase 1.1 (próximas 4-6 semanas): 1. Lambda Query + API Gateway con citación obligatoria 2. Emitir métricas custom CloudWatch desde el código 3. Backend remoto de Terraform (S3 + DynamoDB lock) 4. CI/CD GitHub Actions 5. Gold-set de 20 queries × 4 verticales para Recall@5

Fase 2 (Año 2 — Capa Agente LLM): - LLM generativo (Bedrock Claude / Nova) conectado al índice Aurora - Instrumentación de Capa 4 KPIs (Task Success Rate, Faithfulness, Hallucination Rate, etc.) - Bedrock Guardrails + PII detection layer - Expansión a 8 verticales + integración con Banco Acme + particionado Aurora por `version_id`